Codage de Huffman

Yassir Chakir et Anas Sabir *Équipe EF07, Sciences du Numérique, 2024-2025*

Résumé

Ce rapport vise à présenter la méthode adoptée pour résoudre divers problèmes rencontrés au cours de ce projet. En particulier, il traite de la représentation de l'arbre de Huffman et de sa table, ainsi que de la vérification de la possibilité de coder tous les caractères en moins d'un octet. Il détaille également la manière dont les codes des caractères sont représentés. Tout au long du document, nous explorerons la conception des algorithmes utilisés, les choix techniques effectués, et la démarche mise en œuvre. Le rapport se conclut par un bilan technique et une réflexion personnelle.

1 Introduction

Le codage de Huffman est une méthode de compression sans perte qui réduit efficacement la taille des fichiers en assignant des codes binaires de longueur variable en fonction des fréquences des caractères. Ce projet a pour objectif de développer une application capable de compresser un fichier texte tout en conservant la possibilité de le décompresser ultérieurement.

2 Architecture modulaire

Le programme est structuré autour de plusieurs modules clés interconnectés pour assurer la compression et la décompression efficaces des fichiers.

- **Module Compresser :**
  - Gère les entrées de la ligne de commande.
  - Effectue le calcul des fréquences des caractères, la construction de l'arbre de Huffman, le calcul des codes binaires, et l'encodage du fichier source.
  - Affiche les informations supplémentaires en mode bavard.
- **Module Arbre Binaire :**
  - Gère la construction et la manipulation de l'arbre de Huffman, essentiel pour la génération des codes binaires en fonction des fréquences des caractères.
- **Module Décompresser :**
  - Gère les entrées de la ligne de commande.
  - Effectue la récupération des caractères utilisés, du parcours infixe, et des codes des caractères pour décoder le fichier compressé.

Ces modules collaborent pour garantir une compression efficace des données et une décompression précise, assurant ainsi la traçabilité et l'intégrité des informations.

3 Principaux choix réalisés

- **Représentation de l'arbre :** Utilisation d'un arbre binaire parfait pour stocker les fréquences des caractères et leurs codes.
- **Constructions des noeuds sans tri séparé :** Plutôt que d'utiliser un module de tri séparé, nous avons choisi de trouver directement les deux plus petites fréquences et de les permuter avec les deux derniers symboles. Cette méthode réduit considérablement la complexité tout en assurant la construction correcte de l'arbre de Huffman.
- **Format des fichiers compressés :** Intégration des métadonnées : liste des symboles utilisés et la signature de l'arbre en octet en début de fichier, suivies du texte d'origine après compression en octets pour permettre la décompression.
- **Gestion des données binaires :** Les codes sont regroupés par blocs de 8 bits pour être enregistrés sous forme d'octets.
- **Stockage des caractères pour la décompression :** Un tableau est utilisé pour conserver les caractères employés dans le codage. Cela permet de garantir l'ordre d'apparition des caractères lors du parcours infixe de l'arbre.
- **Utilisation d'un `Unbounded_String` :** Pour stocker le parcours infixe du fichier compressé, un `Unbounded_String` est utilisé. Ce type de donnée est particulièrement pratique pour la manipulation, car il permet de stocker dynamiquement les octets lus séquentiellement, rendant le processus de décompression plus fluide.

4 Algorithmes

4.1 Construction de l'arbre de Huffman

L'arbre est construit en fusionnant les deux nœuds ayant les fréquences les plus faibles. Ce processus est répété jusqu'à obtenir un seul arbre racine.

Algorithm 1: Construction de l'arbre de Huffman

Input: Table des Noeuds de l'arbre
Output: Racine de l'arbre de Huffman

```
while la taille de la table des noeuds contient plus d'un élément do
    Permuter les deux derniers éléments de la table avec les deux éléments ayant
      les plus petites fréquences;
    Fusionner les deux éléments ayant les plus petites fréquences;
    Créer un nouveau nœud avec une fréquence égale à la somme des fréquences
      des deux éléments fusionnés;
    Remplacer ces deux éléments par le nouveau nœud dans la table;
    Réduire la taille de la table des fréquences d'un élément;
```

4.2 Codage de chaque symbole à partir de l'arbre de Huffman

L'algorithme suivant traverse l'arbre de Huffman pour générer le code binaire de chaque symbole. Chaque feuille de l'arbre correspond à un symbole, et le chemin vers cette feuille détermine son code binaire.

Algorithm 2: Codage des symboles à partir de l'arbre de Huffman

Input: Arbre de Huffman, code_courant, Tableau des codes (Tableau_Octet)
Output: Tableau des codes avec chaque symbole et son code binaire

```
Procédure Codage(Arbre, code_courant, Tableau_Octet) :
    if Arbre.Sous_Arbre_Gauche = null et Arbre.Sous_Arbre_Droit = null then
        Ajouter le symbole de la feuille et son code_courant dans Tableau_Octet;
    if Arbre.Sous_Arbre_Gauche ≠ null then
        Appeler Codage(Arbre.Sous_Arbre_Gauche, code_courant + "0",
          Tableau_Octet);
    if Arbre.Sous_Arbre_Droit ≠ null then
        Appeler Codage(Arbre.Sous_Arbre_Droit, code_courant + "1",
          Tableau_Octet);
```

4.3 Parcours infixe de l'arbre pour créer le code de l'arbre et la liste des symboles en octets

L'algorithme ci-dessous effectue un parcours infixe de l'arbre de Huffman pour générer le code de l'arbre et remplir une liste des symboles encodés en octets.

Algorithm 3: Parcours infixe de l'arbre pour le codage et la liste des symboles

Input: Arbre de Huffman, Code_arbre, Tableau des symboles (TabDesSymboles)
Output: Code de l'arbre, Liste des symboles en octets

```
Procédure Parcours_Infixe(Arbre, Code_arbre, TabDesSymboles) :
    if Arbre.Sous_Arbre_Gauche ≠ null then
        Code_arbre ← Code_arbre + "0";
        Appeler Parcours_Infixe(Arbre.Sous_Arbre_Gauche, Code_arbre,
          TabDesSymboles);
    if Arbre.Sous_Arbre_Droit ≠ null then
        Code_arbre ← Code_arbre + "1";
        Appeler Parcours_Infixe(Arbre.Sous_Arbre_Droit, Code_arbre,
          TabDesSymboles);
    if Arbre.Sous_Arbre_Gauche = null et Arbre.Sous_Arbre_Droit = null et
      Arbre.Cle ≠ "$" then
        Incrémenter la taille de TabDesSymboles;
        Ajouter le symbole correspondant à Arbre.Cle dans TabDesSymboles;
    if Arbre.Cle = "$" then
        Incrémenter la taille de TabDesSymboles;
        Ajouter -1 dans TabDesSymboles;
```

4.4 Création du fichier `File_Name.hff`

L'algorithme suivant permet de créer un fichier compressé à partir de l'arbre de Huffman et des symboles encodés.

Algorithm 4: Création du fichier `File_Name.hff`

Input: Nom du fichier (`File_Name`), Tableau des symboles (`TabDesSymboles`), Tableau des octets (`Tableau_Octet`), Taille (`Size`), Code de l'arbre (`Code_arbre`)

Output: Fichier compressé `File_Name.hff`

```
Procédure Creation(File_Name, TabDesSymboles, Tableau_Octet, Size, Code_arbre) :
    Créer le fichier File_Name.hff en mode sortie;
    Initialiser le flux de sortie S;
    S1 ← Affichage du texte à partir de Tableau_Octet et Size;
    S2 ← Code_arbre + "1";
    for chaque symbole dans TabDesSymboles do
        Écrire le symbole en octet dans le flux S;
    Calculer la longueur de S2 modulo 8;
    if le reste ≠ 0 then
        Compléter S2 avec des "0" jusqu'à ce que sa longueur soit un multiple de 8;
    for chaque groupe de 8 bits dans S2 do
        Convertir le groupe de bits en octet et l'écrire dans le flux S;
    for chaque octet dans Tableau_Octet correspondant au symbole spécial "$" do
        Ajouter la valeur associée à l'octet à S1;
    Calculer la longueur de S1 modulo 8;
    if le reste ≠ 0 then
        Compléter S1 avec des "0" jusqu'à ce que sa longueur soit un multiple de 8;
    for chaque groupe de 8 bits dans S1 do
        Convertir le groupe de bits en octet et l'écrire dans le flux S;
    Fermer le fichier File_Name.hff;
```

4.5 Affichage de l'arbre de Huffman

L'arbre de Huffman peut être affiché de manière récursive pour visualiser sa structure. La procédure suivante montre comment afficher chaque nœud de l'arbre avec son indice et sa valeur, en utilisant une indentation pour marquer les niveaux de profondeur.

Algorithm 5: Affichage de l'arbre de Huffman

```
Input: Arbre de Huffman
Output: Affichage de l'arbre
if l'arbre est vide then
    Afficher "L'arbre est vide";
    Retourner;
Afficher "(" et la valeur du nœud racine;
Afficher_arbre_bis(arbre, "", "");
Procédure afficher_arbre_bis(arbre, prefixe, indent) : if l'arbre a un sous-arbre gauche then
    Afficher "–0–" et la valeur du sous-arbre gauche;
    afficher_arbre_bis(arbre.sous_arbre_gauche, prefixe, indent  "| ");
if l'arbre a un sous-arbre droit then
    Afficher "–1–" et la valeur du sous-arbre droit;
    afficher_arbre_bis(arbre.sous_arbre_droit, prefixe, indent  " ");
```

4.6 Conversion d'un Entier en Chaîne de 8 Bits

La fonction `OctetTo8Bits` convertit un entier donné en une chaîne de 8 bits, en représentant chaque bit de l'entier sous forme binaire.

Algorithm 6: Conversion d'un entier en une chaîne de 8 bits

```
Input: n : Entier
Output: Chaîne de 8 bits
Fonction 8Bitsn :
    S ← Chaîne vide;
    copie ← n;
    for i ← 1 to 8 do
        if copie ≥ 2^(8−i) then
            S ← S + "1";
            copie ← copie − 2^(8−i);
        else
            S ← S + "0";
    return Les 8 premiers caractères de S;
```

4.7 Lecture du Fichier Binaire et Extraction des Données

La procédure `Lire` lit un fichier binaire donné, extrait un tableau de symboles, et obtient une chaîne contenant le code de l'arbre de Huffman et le texte compressé.

Algorithm 7: Lecture du fichier binaire et extraction des données

```
Input: File_Name : Nom du fichier binaire
Output: Tableau : Tableau des symboles, S1 : Chaîne contenant le code de
        l'arbre et le texte
Procédure Lire(File_Name, Tableau, S1) :
    nb ← 1;
    char_finish ← faux;
    Ouvrir le fichier File_Name;
    S ← Flux du fichier;
    while fin du fichier non atteinte do
        Octet ← Lire un octet à partir de S;
        if char_finish = faux then
            Tableau[nb] ← Entier de Octet;
            if nb > 1 et Tableau[nb] = Tableau[nb − 1] then
                Tableau[nb] ← −1;
                char_finish ← vrai;
            nb ← nb + 1;
        if char_finish then
            S1 ← S1 + OctetTo8Bits(Octet);
    S1 ← Sous-chaîne de S1 à partir du 9ème caractère jusqu'à la fin;
    Fermer le fichier;
```

4.8 Extraction des Symboles et Séparation du Code de l'Arbre et du Texte

La procédure `extract_symbols` identifie les codes de chaque symbole à partir de l'arbre de Huffman et sépare le code de l'arbre du texte compressé.

4.9 Extraction des Symboles et Séparation du Code de l'Arbre et du Texte

La procédure `extract_symbols` identifie les codes de chaque symbole à partir de l'arbre de Huffman et sépare le code de l'arbre du texte compressé.

Algorithm 8: Extraction des symboles et séparation du code de l'arbre et du texte

Input: *Symbols* : Tableau des symboles, S : Chaîne contenant le code de l'arbre et le texte, *nb_chars* : Nombre de symboles

Output: *code_chars* : Tableau des couples (symbole, code Huffman)

```
Procédure extract_symbols(Symbols, S, code_chars, nb_chars) :
    indice_dollar ← Symbols[1];
    indice_char ← 1;
    indice ← 1;
    symbol_courant ← Unbounded_String("");
    while indice_char ≤ nb_chars do
        symbol_courant ← symbol_courant + S[indice];
        if S[indice + 1] =′ 1′ then
            if indice_char = indice_dollar then
                code_chars[indice_char].Cle ← Unbounded_String("$");
            else
                s1[1] ← Character'Val(Symbols[indice_char]);
                if indice_char < indice_dollar then
                    s1[1] ← Character'Val(Symbols[indice_char + 1]);
                code_chars[indice_char].Cle ← Unbounded_String(s1);
            code_chars[indice_char].Valeur ← symbol_courant;
            indice_char ← indice_char + 1;
            if indice_char ≤ nb_chars then
                while symbol_courant[length(symbol_courant)] ≠′ 0′ do
                    symbol_courant ←
                     symbol_courant[1..length(symbol_courant) − 1];
                symbol_courant ←
                 symbol_courant[1..length(symbol_courant) − 1];
        indice ← indice + 1;
    if indice mod 8 ≠ 0 then
        indice ← indice + 8 − (indice mod 8);
    S ← S[indice + 1..length(S)];
```

Algorithm 9: Extraction des symboles et séparation du code de l'arbre et du texte

Input: *Symbols* : Tableau des symboles, S : Chaîne contenant le code de l'arbre et le texte, *nb_chars* : Nombre de symboles

Output: *code_chars* : Tableau des couples (symbole, code Huffman)

```
Procédure extract_symbols(Symbols, S, code_chars, nb_chars) :
    indice_dollar ← Symbols[1];
    indice_char ← 1;
    indice ← 1;
    symbol_courant ← Unbounded_String("");
    while indice_char ≤ nb_chars do
        symbol_courant ← symbol_courant + S[indice];
        if S[indice + 1] =′ 1′ then
            if indice_char = indice_dollar then
                code_chars[indice_char].Cle ← Unbounded_String("$");
            else
                s1[1] ← Character'Val(Symbols[indice_char]);
                if indice_char < indice_dollar then
                    s1[1] ← Character'Val(Symbols[indice_char + 1]);
                code_chars[indice_char].Cle ← Unbounded_String(s1);
            code_chars[indice_char].Valeur ← symbol_courant;
            indice_char ← indice_char + 1;
            if indice_char ≤ nb_chars then
                while symbol_courant[length(symbol_courant)] ≠′ 0′ do
                    symbol_courant ←
                      symbol_courant[1..length(symbol_courant) − 1];
                symbol_courant ←
                  symbol_courant[1..length(symbol_courant) − 1];
        indice ← indice + 1;
    if indice mod 8 ≠ 0 then
        indice ← indice + 8 − (indice mod 8);
    S ← S[indice + 1..length(S)];
```

5 Types de données

Les types utilisés dans l'implémentation sont définis comme suit :

5.1 Type T_Element

Ce type représente un élément de base, constitué d'un caractère `Cle` et de sa fréquence d'apparition `Frequence` dans le texte. Ce type est utilisé pour stocker les informations nécessaires à la construction de l'arbre de Huffman.

```
type T_Element is
    record
        Cle : Character;
        Frequence : Integer;
    end record;
```

5.2 Type T_Tab

Il s'agit d'un tableau de 256 éléments de type `T_Element`, utilisé pour représenter les fréquences des 256 caractères possibles dans un fichier texte. Ce tableau est essentiel pour la construction de l'arbre de Huffman.

```
type T_Tab is array(1..256) of T_Element;
```

5.3 Type T_Noeud

Ce type définit un nœud de l'arbre de Huffman. Chaque nœud contient un caractère `Cle`, une valeur `Valeur` qui est la somme des fréquences des nœuds enfants, ainsi que deux pointeurs vers les sous-arbres gauche et droit.

```
type T_Noeud is
    record
        Cle : Character;
        Valeur : Integer;
        Sous_Arbre_Gauche : T_ABR;
        Sous_Arbre_Droit : T_ABR;
    end record;
```

5.4 Type T_ABR

Ce type est un pointeur qui permet de manipuler dynamiquement les nœuds de l'arbre binaire. Il est utilisé pour accéder aux nœuds de l'arbre de Huffman.

```
type T_ABR is access T_Noeud;
```

5.5 Type T_Couple

Ce type définit une paire de valeurs : une chaîne de caractères `Cle` et une autre chaîne `Valeur` qui représente un code Huffman associé à cette clé.

```
type T_Couple is
    record
        Cle : Unbounded_String;
        Valeur : Unbounded_String;
    end record;
```

5.6 Type T_Tab_Octet

Il s'agit d'un tableau de 256 éléments de type `T_Couple`, représentant les codes Huffman de chaque caractère.

```
type T_Tab_Octet is array(1..256) of T_Couple;
```

5.7 Type T_Tab_Abr

Il s'agit d'un tableau d'arbres binaires de Huffman (pointeurs de type `T_ABR`), utilisés pour stocker plusieurs arbres de Huffman.

```
type T_Tab_Abr is array(1..256) of T_ABR;
```

6 Démarche de test

Pour garantir la fiabilité, plusieurs niveaux de tests ont été effectués :

- **Tests unitaires :** Validation individuelle des modules de compression et d'arbre binaire.
- **Tests d'intégration :** Vérification de la compatibilité entre les modules en compressant des fichiers variés.
- **Cas limites :** Évaluation avec des fichiers contenant peu ou beaucoup de caractères uniques.

7 Difficultés rencontrées et solutions

- **Création de l'arbre : gestion des symboles déjà traités :** Une difficulté initiale concernait la répétition des symboles dans l'arbre de Huffman. Les symboles déjà traités réapparaissaient dans la structure, ce qui perturbait la construction correcte de l'arbre. **Solution** : Exclure explicitement les éléments déjà utilisés en les déplaçant vers la fin du tableau des fréquences, tout en réduisant la taille effective de ce tableau à chaque itération. Cela a permis de limiter les opérations uniquement aux éléments restants.
- **Complexité de l'affichage de l'arbre :** Lors de l'implémentation pour l'option "bavard", une difficulté majeure résidait dans le respect de la forme d'affichage demandée pour représenter l'arbre de Huffman. Cette option exige une sortie détaillée et lisible montrant les étapes successives de la construction de l'arbre, ce qui s'est avéré complexe à gérer.

8 Organisation de l'équipe

La répartition des tâches entre les membres de l'équipe est la suivante :

Membre	**Tâches**
Yassir Chakir	Création de l'arbre de Huffman, Chargement de l'arbre de Huffman, Encodage du texte et création du fichier compressé, Reconstruction des codes associés aux caractères à partir de la signature de l'arbre,Implémentation de la procedure d'affichage de l'arbre , Décodage du texte et création du fichier décompressé
Sabir Anas	Calcul des fréquences d'apparition des caractères, Création du parcours infixe, Création des codes associés aux caractères, Traitement des exceptions, Récupération du parcours inxe, Récupération de la signature de l'arbre, Récupération du texte compressé à décoder

Table 1 – Répartition des tâches au sein de l'équipe

9 Bilan technique et perspectives

Le projet a atteint ses objectifs principaux, mais des améliorations peuvent être envisagées :

- **Optimisation de la gestion des données :** Réduction de la taille des métadonnées dans le fichier compressé.
- **Performance :** Utilisation d'une structure de tas pour accélérer la construction de l'arbre.

10 Bilan personnel

10.1 Yassir Chakir

J'ai apprécié travailler sur ce projet qui m'a permis de renforcer mes compétences en structures de données.

10.2 Anas Sabir

Ce projet a été une expérience très enrichissante, en particulier dans la gestion des fichiers binaires et le travail sur des structures de données complexes, telles que les arbres.